Karix WhatsApp Integration — Requirements

Purpose: this document lists everything needed **from Karix** (account setup, API documentation, template approvals, webhook configuration) to switch the GoMechanic Tech Support system's WhatsApp notifications from "logged but not sent" to fully live. Share this with your Karix account manager / technical contact, or use it as a checklist while going through the Karix dashboard yourself.

Current state: the integration code is fully built and every trigger point is wired up (see "Where this plugs in" below), but nothing has actually been sent yet — every attempt is logged as `SKIPPED_NOT_CONFIGURED` because none of the items below have been supplied. Nothing in the ticket workflow depends on WhatsApp actually sending, so this can be completed at any time without touching the rest of the app.

---

1. Account credentials

Item	What it's for	Where it goes
API key / auth token	Authenticates every API call to Karix	`KARIX_API_KEY` in `backend/.env`
Source address (WABA number)	The WhatsApp Business number messages are sent *from*	`KARIX_SOURCE_ADDRESS` in `backend/.env`
API base URL	Karix's API host — sandbox vs. production may differ	`KARIX_API_BASE_URL` in `backend/.env` (currently defaults to `https://api.karix.io`)

Questions for Karix: - Is there a separate sandbox/test environment with its own base URL and credentials, so we can test before going live? - If testing in sandbox, is there a list of test phone numbers we need to allowlist first?

---

2. API documentation — sending a template message

The integration currently guesses at Karix's request/response shape (see `backend/apps/notifications/karix_client.py::_build_payload`). We need the **actual, current** API reference for sending a WhatsApp template message, specifically:

- Full endpoint path and HTTP method (e.g. `POST /whatsapp/v1/message`)
- Authentication scheme — is the API key a `Bearer` header, a custom header name, or a query param?

- Exact request body shape: how the destination number, source number, template ID, and template variables/placeholders are named and nested
- Whether template variables are positional (`{{1}}`, `{{2}}`...) or named
- The success response shape — specifically, what field contains the provider's message ID (used for delivery tracking)
- Full list of error response codes/shapes (invalid number, template not approved, user opted out, rate limited, etc.) so we can surface useful errors instead of a generic failure
- Any rate limits (messages/second, messages/day) we should respect

---

3. WhatsApp template approval (8 templates needed)

WhatsApp Business requires every outbound template to be **pre-approved by Meta** before it can be sent. These 8 templates need to be created and approved (via Karix's dashboard or however Karix handles Meta template submission), and we need the resulting **template ID** for each — that ID gets entered into the app's Configuration page once approved.

Key	Triggered when	Suggested variables available at send time
`TICKET_CREATED`	A ticket is raised (by a logged-in user or the public no-login form)	Ticket ID, title, platform
`TICKET_ASSIGNED`	A ticket is assigned/reassigned to an engineer	Ticket ID, assigned engineer's name
`TICKET_IN_PROGRESS`	Status moves to "In Progress"	Ticket ID
`INFO_REQUIRED`	Status moves to "Waiting for User Response"	Ticket ID
`TICKET_RESOLVED`	Status moves to "Resolved"	Ticket ID
`CSAT_CHECK`	Fired immediately after `TICKET_RESOLVED`	Ticket ID — **needs interactive reply buttons**, see below
`TICKET_REOPENED`	User replies "not satisfied" (or staff records that manually)	Ticket ID
`TICKET_CLOSED`	User replies "satisfied" (or staff records that manually)	Ticket ID

Questions for Karix: - What's the template submission process — do we submit copy through Karix, or directly through a Meta Business Manager account? - Typical approval turnaround time? - Template category requirements — are these "Utility" category (transactional/ account-related), since none are

marketing? Utility templates typically have a much simpler approval bar and no marketing opt-in requirements. - **Does Karix support interactive** reply-button **templates** (e.g. two buttons: "Satisfied" / "Not Satisfied") for `CSAT_CHECK`? This is the ideal UX for that message — the user taps a button instead of typing free text. If not supported, we'll fall back to asking them to reply with the words "Satisfied" / "Not Satisfied" (already implemented as a fallback parser, see §5). - Character limits, supported variable count, and any restrictions on variable content (e.g. no special characters) for template bodies.

---

4. Inbound webhook (for receiving the CSAT reply)

After `CSAT_CHECK` is sent, the user's reply (button tap or free text) needs to reach our backend to auto-close or reopen the ticket. Our endpoint is already built and ready to receive this:

```
POST https://<your-backend-host>/api/notifications/webhook/karix
```

What we need from Karix to finish wiring this up:

- **Exact inbound webhook payload shape.** Our current implementation assumes a JSON body of `{"source": "<sender phone>", "text": "<message body>"}` — this is a placeholder guess, not verified against real Karix docs. We need the actual field names for: sender's phone number, message text, and (if interactive buttons are supported) the button payload/ID field, plus the inbound message ID and timestamp fields.
- **How to register this webhook URL** — is it configured in a Karix dashboard setting, via an API call, or provided to a Karix account rep?
- **Signature / authentication scheme for verifying inbound webhook calls are genuinely from Karix** (e.g. an HMAC signature header with a shared secret, a static bearer token, or an IP allowlist). **This is currently unimplemented** — the endpoint has no verification yet and will need one before going live, since anyone who knows the URL could currently POST to it.
- Does Karix retry webhook delivery on failure (e.g. if our server is briefly down)? What's the retry/backoff policy?

---

5. Behavior already built on our side (for context)

So Karix/your team knows what to expect without needing to read our code:

- If `CSAT_CHECK` gets a reply, we interpret it as satisfied or not satisfied by matching text like `"satisfied"`, `"yes"`, `"resolved"` (→ closes the ticket) vs. `"not satisfied"`, `"not resolved"`, `"no"` (→ reopens it), case-insensitive. If Karix supports button-based replies, we'd much rather match an exact button payload/ID than parse free text — let us know the payload shape and we'll switch to that.
- We match an inbound reply to a ticket by looking up the **most recent ticket in "Resolved" status awaiting a CSAT reply for that phone number** — so the sender's phone number in the

webhook payload must match the number the ticket was originally raised with.

- Every send attempt (success, failure, or skipped) is logged internally regardless of outcome — nothing about the ticket workflow depends on WhatsApp succeeding.

---

6. Compliance / policy questions

- Are ticket-raisers (who submit their phone number when raising a ticket, either via account signup or the public no-login form) considered sufficiently opted-in to receive these transactional WhatsApp messages, or does Karix/Meta require an additional explicit opt-in step first?
- WhatsApp's standard 24-hour customer service session window — do these transactional/utility templates fall outside that restriction (i.e. can we message a user any time after they raise a ticket, not just within 24h of their last message)? This matters most for `TICKET_RESOLVED` / `CSAT_CHECK`, which may fire well after the user's last message.
- Any data retention requirements on our end for message logs (we currently keep a full log of every send attempt indefinitely in our own database).

---

7. Cost

- Per-message pricing (utility template vs. any other category, if relevant), and whether inbound replies are billed separately from outbound sends.

Once we have all of the above

1. Set `KARIX_API_KEY`, `KARIX_SOURCE_ADDRESS`, and (if different from default) `KARIX_API_BASE_URL` in `backend/.env`.
2. For each of the 8 templates, set its `karix_template_id` via the Configuration page (WhatsApp Templates tab) or `PATCH /api/config/whatsapp-templates/<id>/`.
3. We update `_build_payload()` in `backend/apps/notifications/karix_client.py` and the webhook field names in `backend/apps/notifications/views.py` to match the real API contract from §2 and §4.
4. We add webhook signature verification per §4.
5. Test end-to-end in Karix's sandbox (if available) before flipping to production credentials.